



PROGRAMAÇÃO FUNCIONAL: Haskell
Listas

Professor Rafael Kingeski

Departamento de Ciência da Computação
Centro de Ciências Tecnológicas - CCT
UDESC - Joinville.

UDESC
UNIVERSIDADE
DO ESTADO DE
SANTA CATARINA

Índice

1. Listas
2. Exemplos
3. Exercícios

Índice

1 Listas

2 Exemplos

3 Exercícios

Listas

Definição: Listas em Haskell são estruturas de dados homogêneas. As listas são estruturas sequenciais.

Compostas de dois segmentos: cabeça e corpo (ou cauda). Em inglês *head* e *tail*.

[1,5,20,4]

Uma lista pode conter qualquer número de elementos, inclusive nenhum.

Uma lista Sem nenhum elemento é chamada de lista vazia. Representada por dois colchetes:

[]

Listas - Operadores

Operador ":" é o operador construtor/separador.

$$1:[] = [1]$$
$$1:2:3:4:5:[] = [1,2,3,4,5]$$

Utilizado para incluir ou separar apenas um elemento a lista.

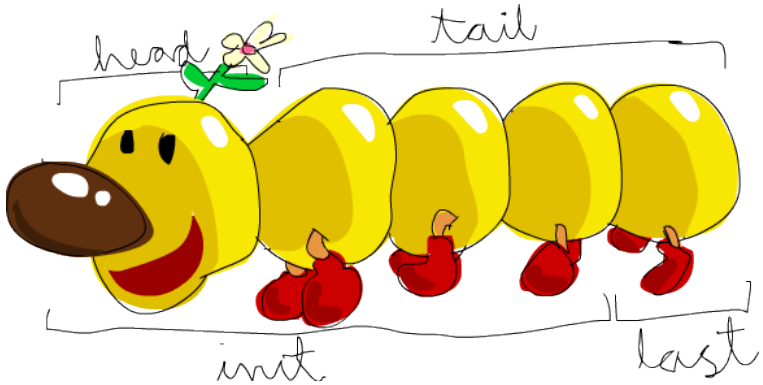
Operador "++" é o operador de concatenação de listas.

$$[1,2,3,4] ++ [5,6,7] = [1,2,3,4,5,6,7]$$

Operador "!!" é o operador de acesso a um elemento da lista pelo seu índice.

$$[2,4,3,5] !! 2 = 3$$

Listas - Funções



head - retorna o valor da cabeça da lista.

```
ghci> head [5,4,3,2,1]
5
```

tail - retorna o corpo da lista.

```
ghci> tail [5,4,3,2,1]
[4,3,2,1]
```

Listas - Funções

last - retorna o último elemento.

```
ghci> last [5,4,3,2,1]
1
```

init - retorna a lista toda exceto o último elemento.

```
ghci> init [5,4,3,2,1]
[5,4,3,2]
```

length - retorna o tamanho da lista.

```
ghci> length [5,4,3,2,1]
5
```

Listas - Funções

reverse - retorna a lista com a ordem dos elementos invertida.

```
ghci> reverse [5,4,3,2,1]
[1,2,3,4,5]
```

maximum e **minimum** - retorna o maior ou menor valor da lista.

```
ghci> minimum [8,4,2,1,5,6]
1
ghci> maximum [1,9,2,3,4]
9
```

sum - soma de todos os elementos da lista.
product - produto de todos os elementos da lista.

```
ghci> sum [5,2,1,6,3,2,5,7]
31
ghci> product [6,2,1,2]
24
ghci> product [1,2,5,6,7,9,2,0]
0
```

Listas - Ranges

Uma lista pode ser definida a partir de um Range.
Define-se uma lista de k até n por:

[k..n]

```
ghci> [1..20]
[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20]
ghci> ['a'..'z']
"abcdefghijklmnopqrstuvwxyz"
ghci> ['K'..'Z']
"KLMNOPQRSTUVWXYZ"
```

Pode-se ainda definir uma lista de k até n por um passo de k,p:

[k,p .. n]

```
ghci> [2,4..20]
[2,4,6,8,10,12,14,16,18,20]
ghci> [3,6..20]
[3,6,9,12,15,18]
```

Listas - Ranges

Pode-se definir uma lista infinita por:

[k ..]

Obs.: Evite utilizar ranges com elementos do tipo Float!

Listas - Compreensão de Lista

Baseado na teoria de conjuntos, podemos definir uma lista pela definição de seus elementos. Por exemplo:

$$A = \{2 \cdot x | x \in N, x \leq 10\}$$

Em Haskell:

```
ghci> [x*2 | x <- [1..10]]
[2,4,6,8,10,12,14,16,18,20]
```

Índice

1 Listas

2 Exemplos

3 Exercícios

Exemplos

Função para calcular o tamanho de uma lista:

```
tamanho [] = 0
tamanho (x:xs) = 1 + tamanho xs
```

Função para calcular a soma de uma lista:

```
soma [] = 0
soma (x:xs) = x + soma xs
```

Função Para Encontrar o maior valor de uma lista.

```
maior [a] = a
maior (a:x) = if (a > maior x) then a else maior x
```

Exemplos

Função que remove todos os valores duplicados de uma lista.

```
remdupli [] = []
remdupli (x:xs) | x `elem` xs = remdupli xs
                | otherwise = x:  remdupli xs
```

Índice

1 Listas

2 Exemplos

3 Exercícios

1. Construa uma lista utilizando compreensão de listas, cujo o conjunto é definido por:

$$A = \left\{ x^3 | x \in N, x \leq 8 \right\}$$

2. Implemente uma função que recebe uma lista e retorna uma nova lista contendo a duplicação dos elementos da lista original.
ex: duplicalist [6, 2, 3] => [6,6,2,2,3,3]

3. Implemente uma função que recebe uma lista de números inteiros e retorne os números ímpares.

Referências

- M. Lipovaca
  Learn You a Haskell for Great Good!
  (O'Reilly, 2011)
- C. C. de Sá; M. F. da Silva
  Haskell: Uma Abordagem Prática
  (Novatec, 2006)